

Projet Computer vision

Détection automatique d'EPI sur chantier

Jonathan White

Sommaire

1. Préparation de la donnée.....	3
1.1 - Réorganisation du dataset	3
1.2 - Exploration et statistiques.....	4
1.3 - Pré-traitement	7
1.4 - Augmentation des données.....	8
2. Choix et configuration du modèle	10
2.1 - Choix du modèle	10
2.2 - Configuration, entraînement et régularisation	10
3. Évaluation	11
4. Inférence	12
Partie 2 : Analyse avancée et déploiement.....	13
2.1 - Axe B : Focus classes EPI	13
2.2 - Axe E : Interface de démonstration (Streamlit).....	14

1. Préparation de la donnée

1.1 - Réorganisation du dataset

Le dépôt SH17 fournit toutes les images dans un seul dossier, avec des fichiers `train.txt` / `val.txt` listant les noms de fichiers par split. Personnellement, j'ai tout téléchargé depuis Kaggle. J'ai réorganisé le dataset au format attendu par YOLO :

dataset_yolo/

images/{train,val,test}/

labels/{train,val,test}/

En lisant les fichiers `train\_files.txt` / `val\_files.txt` et en copiant chaque image et son label correspondant dans le bon dossier (le test étant déduit du reste).

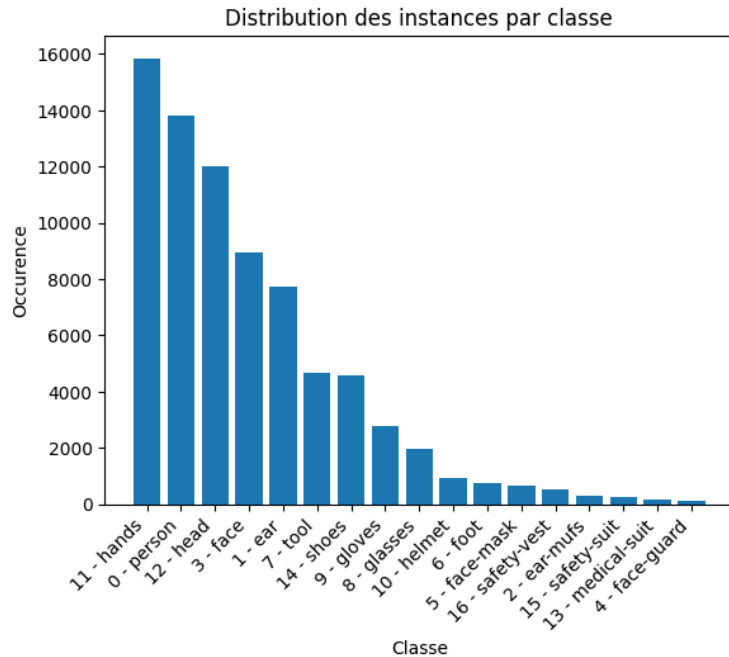
J'ai également généré le fichier `sh17.yaml` nécessaire à YOLO (chemins des dossiers + 17 classes).

Résultat :

```
Création des dossiers terminée
Copie des images/labels terminés
train: 5832 images, 5832 labels
val: 1620 images, 1620 labels
test: 647 images, 647 labels
Fichier yaml créé.
```

1.2 - Exploration et statistiques

Ratio classe majoritaire / minoritaire



La classe majoritaire est HANDS (id=11) et la classe minoritaire est FACE-GUARD(id=4), avec un ratio de 118.3 : la classe majoritaire est représentée 118 fois plus souvent que la minoritaire.

Ce déséquilibre est très important et aura un impact direct sur l'apprentissage : le modèle verra très peu d'exemples des classes rares (face-guard, ear-mufs, safety-suit...) et aura donc beaucoup de mal à les détecter correctement, là où les classes fréquentes (person, hands, head, face...) seront bien apprises.

Visualisation d'exemples annotés (train / val / test)

train - pexels-photo-5409753



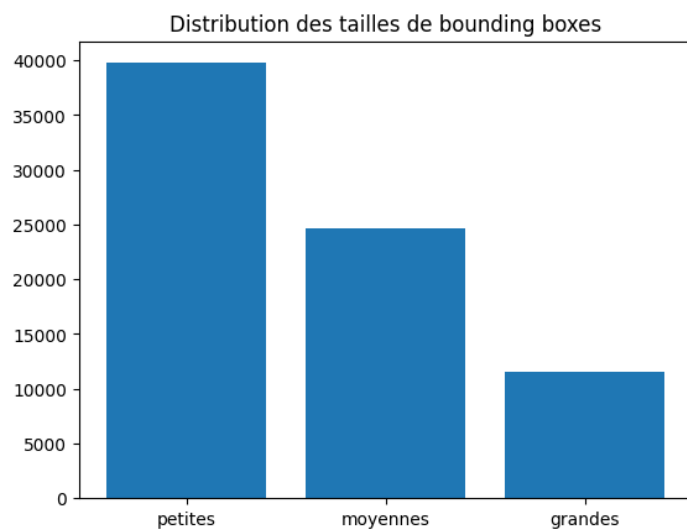
train - pexels-photo-2670327



test - pexels-photo-6098065



Statistiques sur la taille des bounding boxes

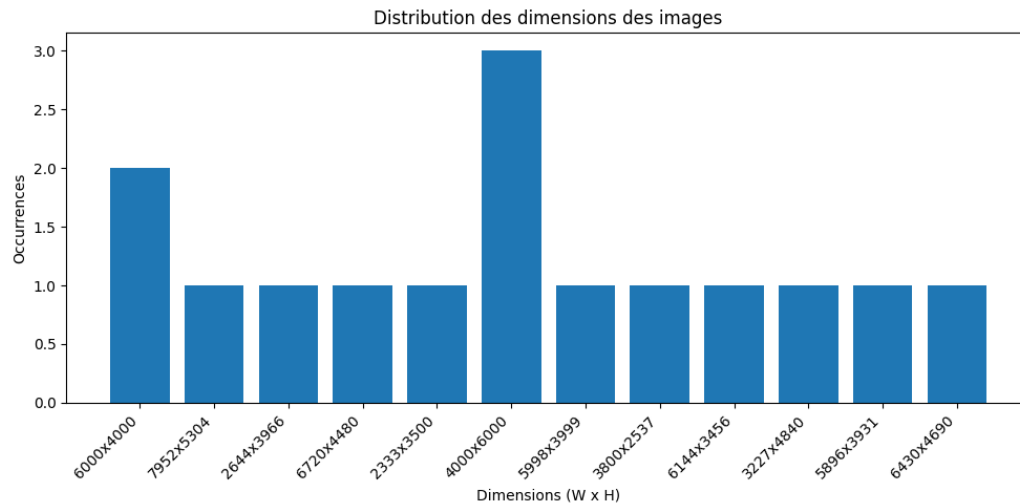


Après recherche, j'ai défini arbitrairement les seuils suivants : une bounding box est considérée comme "petite" si son aire représente moins de 1% de l'image, "moyenne" entre 1% et 10%, et "grande" au-delà.

Ici, on voit qu'il y a énormément de petites boxes. Le modèle aura donc des difficultés à les détecter. Il faudra certainement les recadrer.

1.3 - Pré-traitement

Redimensionnement



Les images du dataset n'ont pas toutes la même résolution d'origine.

YOLO redimensionne automatiquement les images à « 640x640 » avec le paramètre « imsz=640 », ce qui correspond à la résolution d'entrée du modèle choisi (YOLO26-nano).

Normalisation

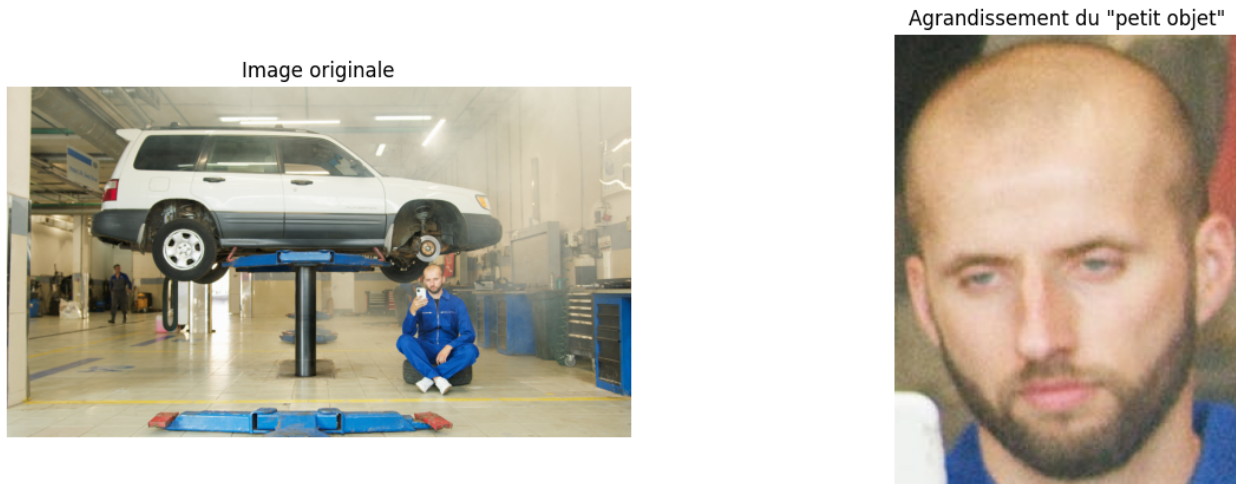
YOLO normalise en interne les pixels de « [0, 255] » vers « [0.0, 1.0] » avant de les passer au réseau. J'ai simplement fait un exemple comme nous l'avons vu dans les TP.

Débruitage

Lors de l'exploration visuelle des images je n'ai constaté aucun artefact. Je n'ai donc appliqué aucun débruitage.

Recadrage (crop) pour les petits objet

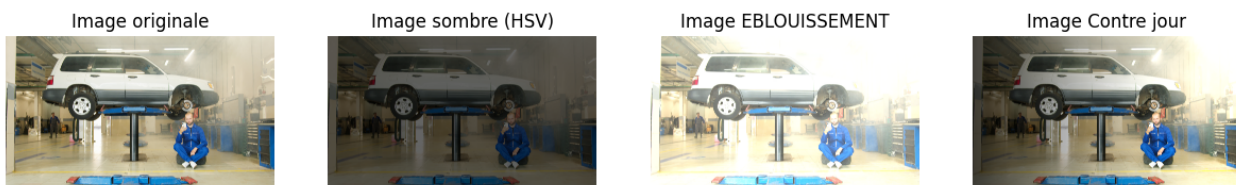
Les statistiques montrent qu'une part importante des bounding boxes sont de petite taille. J'ai illustré le principe d'un recadrage centré sur un petit objet, pour montrer l'intérêt d'un zoom aléatoire sur ce type d'objet. Il est vrai que j'aurais pu faire ça mais d'après la documentation de YOLO, le paramètre « scale » imite ce travail-là. J'ai donc préféré faire confiance à YOLO



1.4 – Augmentation des données

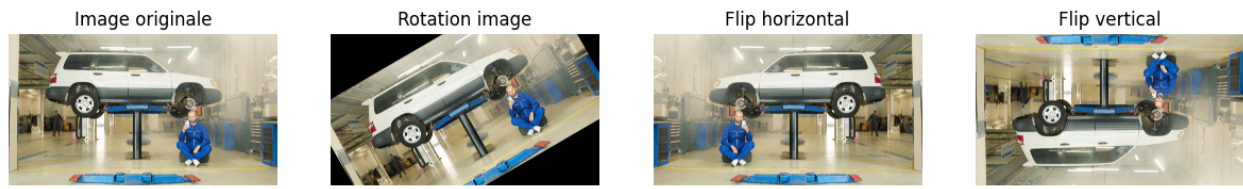
J'ai mis en place trois types d'augmentations :

Variations d'éclairage



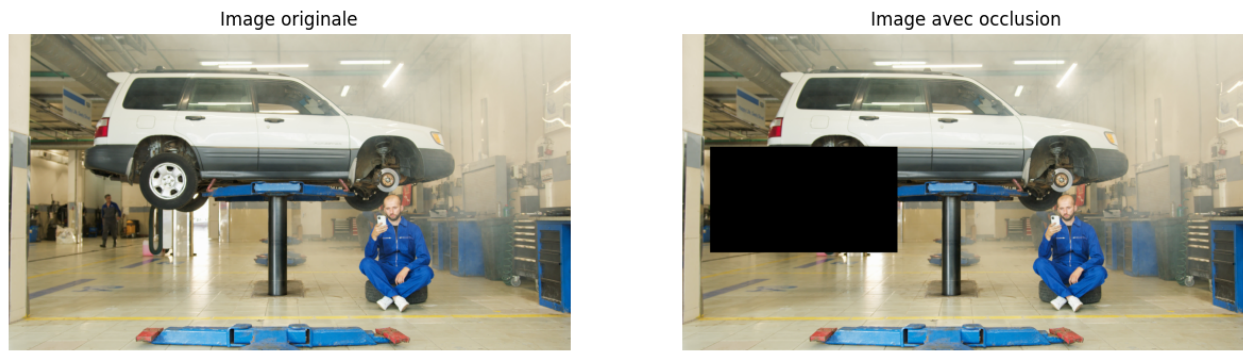
J'ai simulé un assombrissement global (effet nuit, via une réduction de la composante V en HSV comme vu en TP), un éblouissement (ajout d'une valeur constante aux pixels), et un contre-jour (gradient de luminosité appliqué horizontalement sur l'image).

Changements d'angle de vue



J'ai appliqué une rotation de 30°, ainsi que des flips horizontal et vertical, pour simuler différents angles de prise de vue

Occlusions partielles



J'ai simulé une occlusion partielle en plaçant un rectangle noir à une position aléatoire sur l'image.

Lors de l'entraînement final, ces augmentations sont activées via les paramètres YOLO « `fliplr=0.5` » (flip horizontal aléatoire) et « `hsv_v=0.5` » (variation de luminosité aléatoire), qui correspondent directement aux transformations illustrées ci-dessus.

2. Choix et configuration du modèle

2.1 - Choix du modèle

J'ai retenu YOLO car :

- Le projet impose de traiter des flux vidéo en temps réel, ce qui élimine d'office Faster R-CNN.
- Le dataset est fortement déséquilibré : un modèle léger et facile à fine-tuner via transfer learning (YOLO pré-entraîné sur COCO) est un bon choix.
- Pour être franc, c'est celui qu'on a vu encours

2.2 - Configuration, entraînement et régularisation

Architecture retenue

Aucune modification d'architecture n'a été apportée : seule la tête de détection est ré-entraînée pour s'adapter aux 17 classes du dataset

Stratégie de transfer learning

J'ai utilisé les poids pré-entraînés COCO et gelé les 10 premières couches qui correspondent au backbone d'extraction de caractéristiques bas-niveau (contours, textures, formes simples). Ces caractéristiques génériques sont déjà bien apprises sur COCO et transférables à notre tâche.

On laisse les couches profondes (tête de détection) se réentraîner sur les 17 classes du dataset.

Hyperparamètres utilisés

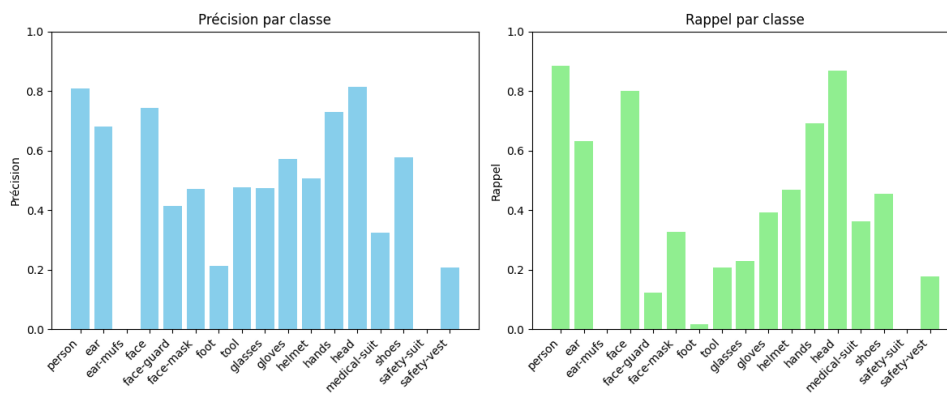
Hyperparamètre	Valeur	Justification
Epochs	20	Compromis temps/qualité sur CPU
Batch size	16	Adaptée à mon CPU
Image size	640	Résolution standard YOLO
Learning rate (lr0)	0,01	Valeur par défaut éprouvée pour le fine-tuning YOLO
Patience (early stopping)	10	Arrête l'entraînement si aucune amélioration de la mAP sur le jeu de validation pendant 10 epochs consécutives
fliplr	0,5	Flip horizontal aléatoire (augmentation de données)
hsv_v	0,5	Variation aléatoire de la luminosité (augmentation de données)

3. Évaluation

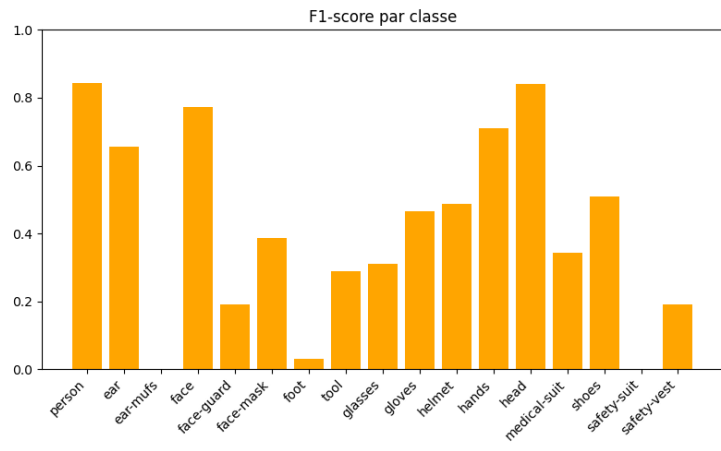
Métriques calculées sur le jeu de test :

- mAP@50 : 0.406
- mAP@50-95 : 0.247

Précision et rappel par classe



F1-score par classe



Dans notre projet, la métrique du RAPPEL est plus adaptée que la précision, car il est plus important de minimiser les faux négatifs (ne pas manquer un travailleur non conforme) que d'éviter les faux positifs (fausses alertes).

On observe ici un rappel global faible et très hétérogène selon les classes. C'est directement lié au déséquilibre du dataset : le modèle n'a tout simplement pas vu assez d'exemples de ces classes rares pour apprendre à les détecter.

4. Inférence

J'ai testé le modèle sur une image du jeu de test. La détection identifie les classes présentes (person, face, hands, head, shoes...). J'ai défini une règle de conformité simple : un travailleur est considéré non conforme si une partie du corps est détectée sans son équipement de protection associé (ex : head -> helmet, person -> safety-vest, face -> lunettes, hands -> gants).

Si une non-conformité est détectée, un bandeau rouge "NON CONFORME - EPI MANQUANT" est affiché.

Bien sûr, cette règle est arbitraire et devrait s'adapter à chaque corps de métier.

J'ai fait la même chose sur une vidéo.

Partie 2 : Analyse avancée et déploiement

2.1 – Axe B : Focus classes EPI

L'objectif de cet axe est de restreindre l'entraînement aux trois classes directement liées à la conformité EPI : helmet (id=10), head (id=12) et safety-vest (id=16).

Pour ça, j'ai créé un nouveau dataset « dataset_yolo_epi » en parcourant tous les fichiers de labels du dataset original, en ne conservant que les lignes correspondant aux classes 10, 12 et 16, et en remappant leurs identifiants : 10 -> 0 (helmet), 12 -> 1 (head), 16 -> 2 (safety-vest).

Un nouveau fichier « sh17_epi.yaml » décrivant ces 3 classes a été généré.

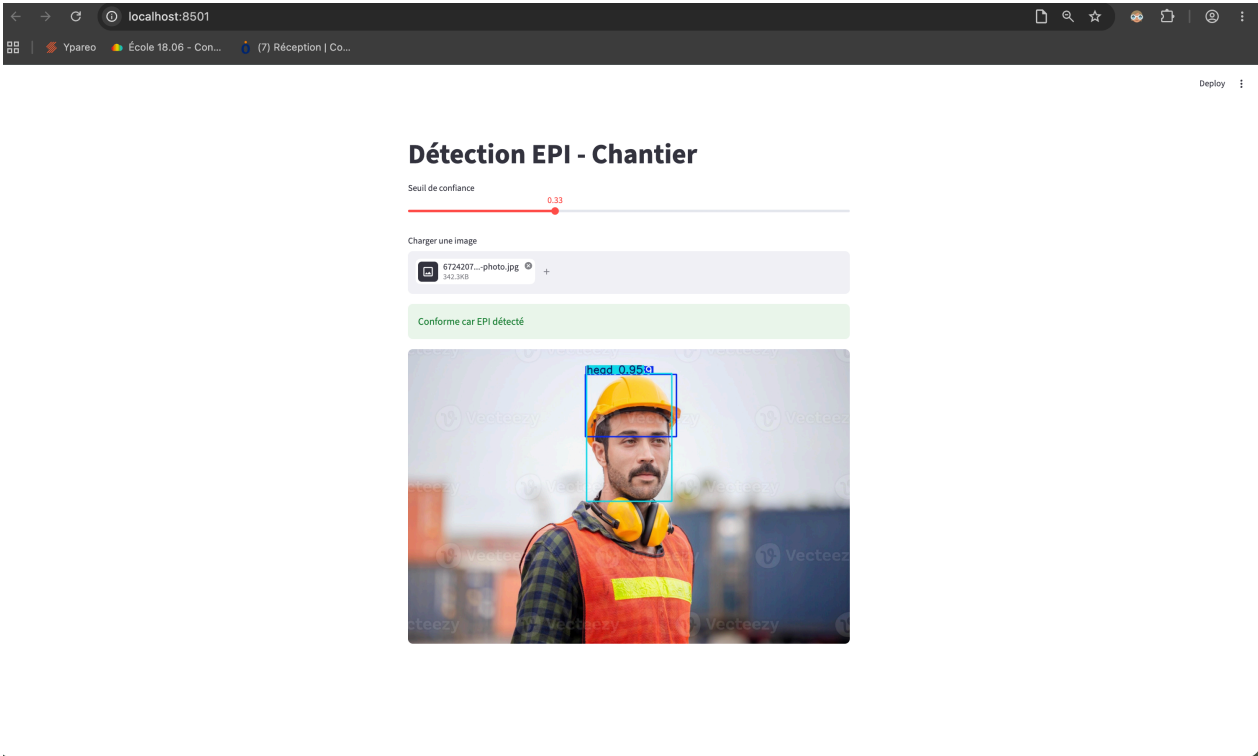
Ensuite, un nouvel entraînement a tourné avec yolo et les mêmes hyperparamètres. Voici les métriques :

```
mAP@50 : 0.406 (17 classes) -> 0.687 (EPI)
mAP@50-95 : 0.247 (17 classes) -> 0.473 (EPI)

Classe| Précision (17cl -> EPI) | Rappel (17cl -> EPI)
helmet | 0.506 -> 0.752 | 0.470 -> 0.506
head | 0.815 -> 0.898 | 0.868 -> 0.893
safety-vest | 0.209 -> 0.542 | 0.176 -> 0.529
```

On voit que la restriction de l'entraînement aux seules classes EPI (helmet, head et safety-vest) améliore les performances du modèle. Ces résultats confirment que la présence de nombreuses classes déséquilibrées dilue la capacité d'apprentissage du réseau.

2.2 - Axe E : Interface de démonstration (Streamlit)



Détection EPI - Chantier

